

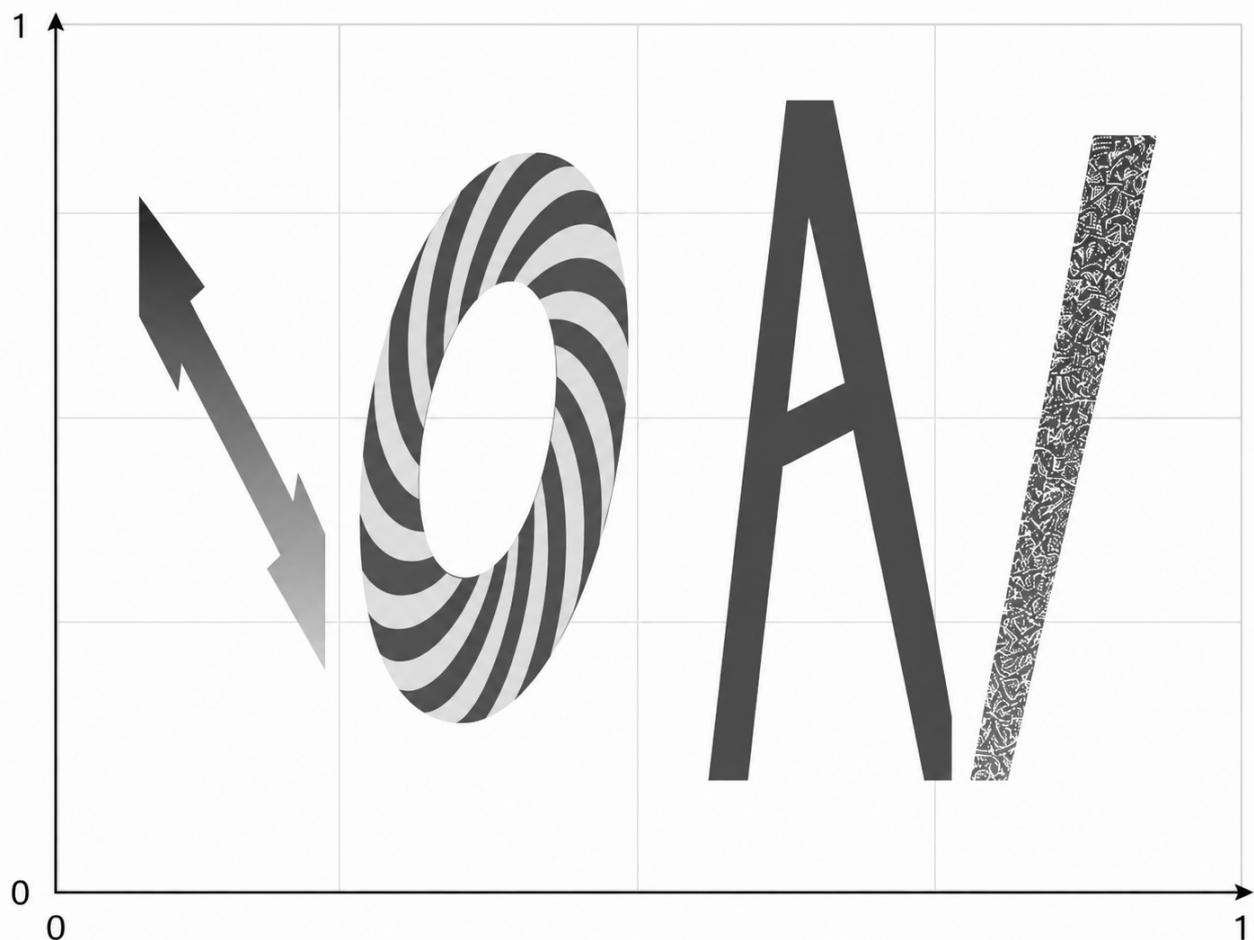
Campo IOAI

- **Limite de tempo:** 5 minutos
- **Armazenamento:** 5 GB
- **Tamanho da solução:** `solution.ipynb`, `custom_model.py` ≤ 1 MB juntos
- **Modelos pré-treinados:** nenhum — treine do zero, sem internet durante a avaliação
- **Pontuação da baseline:** 31.2187

Tarefa

O prefeito de Astana quer decorar a cidade com logotipos estilizados da IOAI. Como estatístico, ele vê tudo — incluindo o logotipo — como uma função espacial $F(x, y, \overline{W})$, em que $x, y \in [0, 1]$ representam coordenadas em um plano 2D e $\overline{W}$ é um conjunto de parâmetros ocultos que definem atributos estilísticos, como as cores e os ângulos das letras.

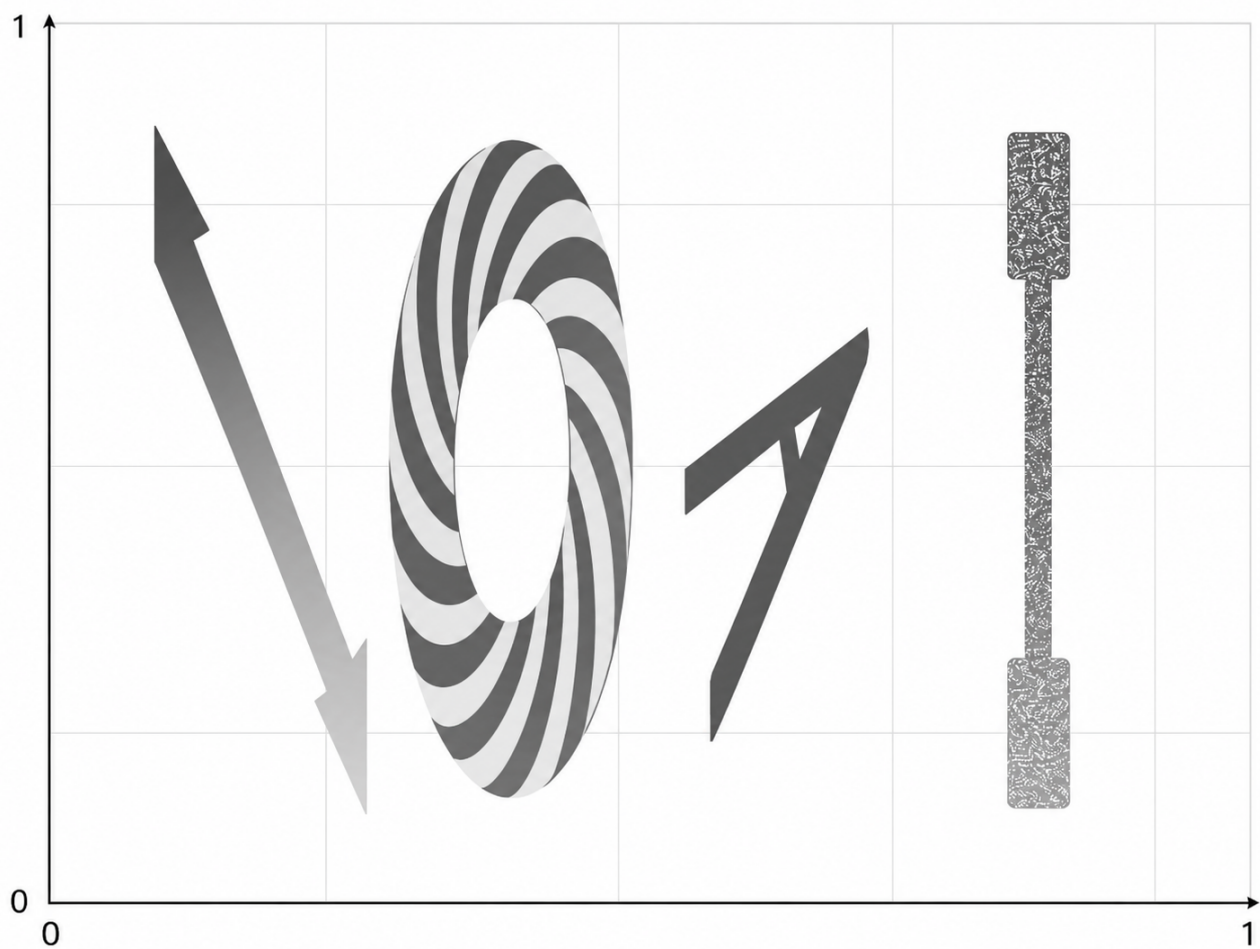
Como F é complexa demais para ser expressa como uma equação matemática explícita, sua tarefa é treinar uma rede neural para aproximá-la. A rede produzirá um valor de **campo IOAI** para qualquer par de coordenadas (x, y) , gerando uma visualização completa do logotipo como mapa de calor ao longo do plano. Eis um exemplo de visualização de F como mapa de calor com alguns parâmetros ocultos específicos $\overline{W}$.

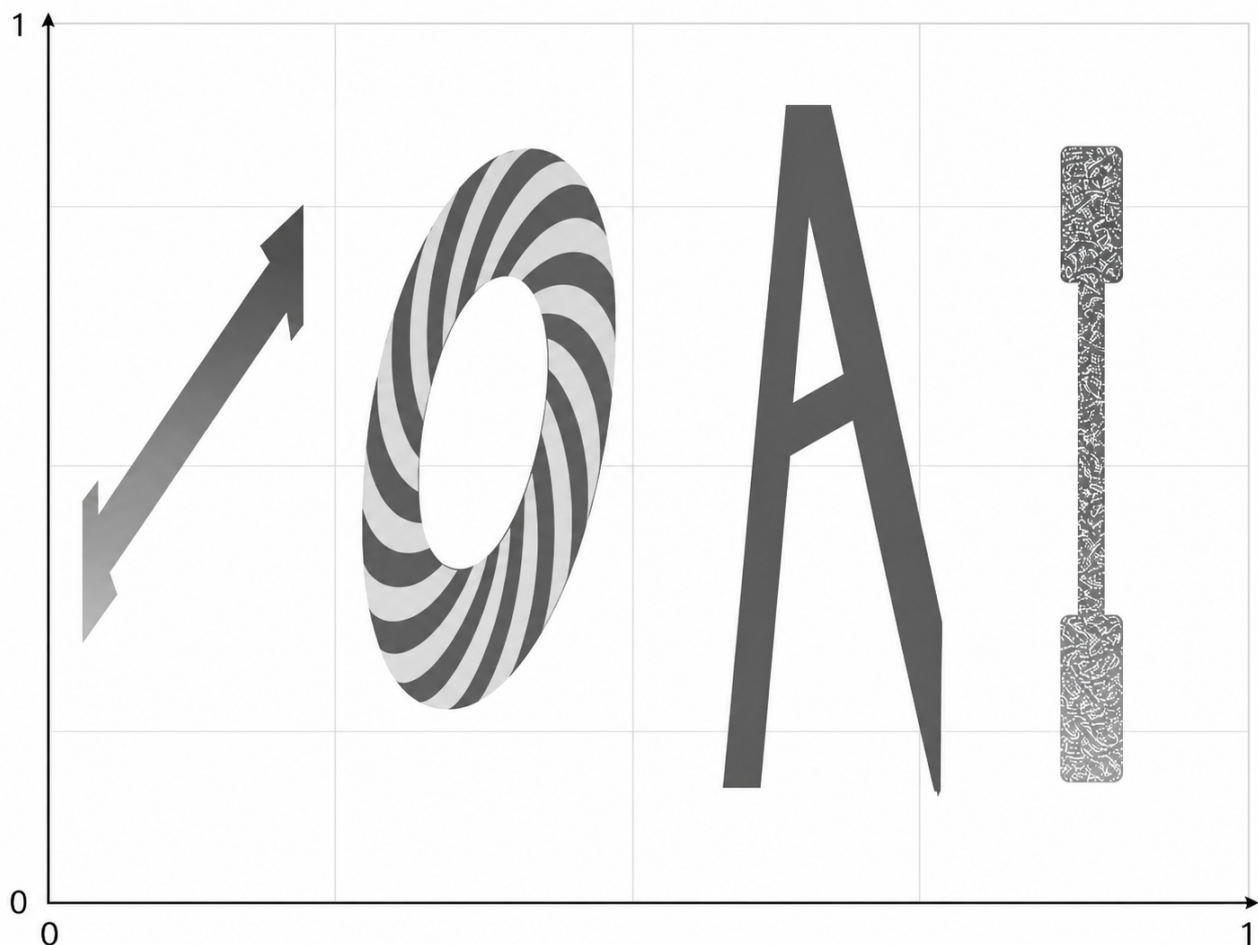


Do que consiste o campo IOAI? Quatro letras e o fundo.

- Os valores dentro da primeira letra $\mathbb{I}$ são muito grandes ($1e+10$ ou mais), com um gradiente linear
- Os valores na letra $\mathbb{O}$ apresentam um padrão espiral
- O valor dentro da letra $\mathbb{A}$ é sempre -1
- Os valores dentro da última letra $\mathbb{I}$ devem ser valores aleatórios do intervalo $[-2026, 2026]$, mesmo se forem avaliados duas vezes no mesmo ponto
- Fora das letras, o valor é sempre zero

A função possui parâmetros ocultos $\overline{W}$, que afetam a escala e a inclinação das letras, juntamente com o intervalo de valores dentro da primeira letra $\mathbb{I}$. No entanto, as letras não se intersectarão. Eis alguns exemplos ilustrativos de como o campo IOAI se apresenta com diferentes $\overline{W}$:





O que é fornecido a você:

Este problema NÃO contém datasets. Em vez disso, é fornecida a função geradora, configurada pelo arquivo de configuração JSON em `data/train_config/field_config.json`.

A configuração de teste está oculta, mas é de natureza semelhante. Sua tarefa é ajustar o modelo ao gerador fornecido usando quantos dados desejar. Suas distribuições de "treino" e "teste" são geradas pelo mesmo gerador — você apenas não sabe em quais pontos (x_i, y_i) será avaliado.

Sua submissão deve consistir em:

- uma classe de modelo de treinamento salva como `custom_model.py`. Esse modelo deve herdar da classe `torch.nn.Module` e usar somente imports de `torch`. Ele deve conter a classe `CustomModel` usada no notebook `solution.ipynb`.
- o notebook `solution.ipynb`, que produzirá os pesos `model.pt`

Pontuação

Para cada região, a pontuação mínima é 0 e a pontuação máxima é 1. A pontuação final é calculada como a média entre todas as cinco regiões (quatro, uma para cada letra, e o fundo) e multiplicada por 100. Há uma **penalidade por parâmetros**:

Se seu modelo tiver mais de 20260 parâmetros, a pontuação será reduzida à metade.

O número de parâmetros é medido por `sum(p.numel() for p in model.parameters())`. Esperamos que seu modelo também opere em modo estocástico, com o `nn.Dropout` do PyTorch fazendo parte do modelo.

Para regiões padrão

Para cada região R (primeira letra `I`, `O`, `A`, `Background`), avaliamos o modelo em $N_R = 512$ pontos de teste (x_i, y_i) com valores verdadeiros v_i e previsões $\hat{v}_i$. Usamos o Erro Absoluto Médio (MAE) normalizado como métrica principal. O MAE é definido como:

$$\text{MAE}(R) = \frac{1}{N_R} \sum_{i=1}^{N_R} |\hat{v}_i - v_i|.$$

E a normalização é realizada como

$$\text{score}(R) = 1 - \min \left(\frac{\text{MAE}(R)}{s_R}, 1 \right),$$

em que $s_R > 0$ é uma constante de escala.

Para a região da última letra `I`

Nesta região, o **dropout é habilitado durante a avaliação**. Para cada ponto de teste j :

1. Executamos o modelo $K = 10$ vezes para obter $\hat{v}_{j,1}, \dots, \hat{v}_{j,K}$.
2. Se alguma saída estiver fora do intervalo $[-2026, 2026]$, então $\text{pointScore}(j) = 0$.
3. Caso contrário, calculamos o desvio padrão σ_j das K saídas e o convertemos em uma pontuação:

$$\text{pointScore}(j) = \min \left(\frac{\sigma_j}{s_E}, 1 \right),$$

em que $s_E > 0$ é uma constante de escala fixa.

A pontuação da região é a média de todos os pontos da região:

$$\text{score}(I_{last}) = \frac{1}{N_E} \sum_{j=1}^{N_E} \text{pointScore}(j),$$

em que $N_E = K * N_R$.

Em termos simples, quanto maior a diversidade, maior será sua pontuação para esta região. **Você não pode usar aleatoriedade em sua forma pura, incluindo as funções `rand*` e `_uniform` do PyTorch; a aleatoriedade deve vir da inferência com o dropout habilitado.**

Como submeter

1. Abra `solution.ipynb` e execute todas as células.
2. Melhore o modelo `CustomModel` em `custom_model.py`
3. Certifique-se de que sua última célula salve seu modelo no arquivo `model.pt`.
4. Na aba `Git` do JupyterLab, rode `stage`, escreva um comentário e faça `commit` de `solution.ipynb` e `custom_model.py` e, em seguida, faça `push`.
5. Retorne à página da competição e clique em **Submit**. O comentário da submissão deve ser igual ao comentário da etapa anterior.